

=== FILE: map_editor/editor.py ===

"""Главный класс редактора"""

```
import os
import json
import pygame
from .config import *
from .ui import Canvas, InfoPanel
from .ui.toolbar import Toolbar
from .ui.tools_panel import ToolsPanel
from .tools import Brush, Eraser
from .tools.selection import Selection
```

```
class MapEditor:
```

```
    """Редактор карт"""
```

```
    def __init__(self, level_file=None):
        pygame.init()
        pygame.display.set_caption("Map Editor – Загрузите уровень")

        self.screen = pygame.display.set_mode((WINDOW_WIDTH, WINDOW_HEIGHT))
        self.clock = pygame.time.Clock()
        self.running = True

        self.dialog_active = False
        self.dialog_choice = None

        self.grid = []
        self.current_file = None
        self.selected_symbol = 'M'
        self.has_changes = False
        self._saving = False

        self._setup_ui()
        self._setup_tools()

        if level_file:
            self.load_level(level_file)
```

```
    def _setup_ui(self):
        # ПАНЕЛЬ ИНСТРУМЕНТОВ (сверху) - ВЫСОТА 50px
        tools_rect = pygame.Rect(0, 0, WINDOW_WIDTH - 250, 50)
        self.tools_panel = ToolsPanel(tools_rect)

        # КАНВАС (под панелью инструментов)
        canvas_rect = pygame.Rect(0, 50, WINDOW_WIDTH - 250, WINDOW_HEIGHT - 50 - 60)
        self.canvas = Canvas(canvas_rect)

        # ИНФОРМАЦИЯ (внизу)
        info_rect = pygame.Rect(0, WINDOW_HEIGHT - 60, WINDOW_WIDTH - 250, 60)
        self.info_panel = InfoPanel(info_rect)

        # ЛЕГЕНДА (справа)
        toolbar_rect = pygame.Rect(WINDOW_WIDTH - 250, 0, 250, WINDOW_HEIGHT)
        self.toolbar = Toolbar(toolbar_rect)
```

```

def _setup_tools(self):
    self.brush = Brush(self)
    self.eraser = Eraser(self)
    self.selection = Selection(self)
    self.current_tool = 'brush'

def _on_change(self):
    if not self._saving:
        self.has_changes = True
        self._auto_save()

def _auto_save(self):
    if self._saving or not self.current_file or not self.grid or not self.has_changes:
        return

    self._saving = True
    self._create_backup()
    self._saving = False

def _create_backup(self):
    if not self.current_file or not self.grid:
        return

    backup_dir = os.path.join(os.path.dirname(self.current_file), 'levels_backup')
    os.makedirs(backup_dir, exist_ok=True)

    base_name = os.path.basename(self.current_file)
    name, ext = os.path.splitext(base_name)
    backup_path = os.path.join(backup_dir, f"{name}_backup{ext}")

    try:
        with open(self.current_file, 'r', encoding='utf-8') as f:
            data = json.load(f)

        data['map'] = self.grid

        with open(backup_path, 'w', encoding='utf-8') as f:
            json.dump(data, f, indent=2, ensure_ascii=False)

    except Exception as e:
        print(f"[Бэкап] Ошибка: {e}")

def _clear_backups(self):
    if not self.current_file:
        return

    backup_dir = os.path.join(os.path.dirname(self.current_file), 'levels_backup')
    if not os.path.exists(backup_dir):
        return

    base_name = os.path.basename(self.current_file)
    name, _ = os.path.splitext(base_name)

    for f in os.listdir(backup_dir):
        if f.startswith(name) and f.endswith('_backup.json'):
            try:
                os.remove(os.path.join(backup_dir, f))
            except:

```

```

        pass

def load_level(self, file_path):
    if not os.path.exists(file_path):
        print(f"[Ошибка] Файл не найден: {file_path}")
        return False

    try:
        with open(file_path, 'r', encoding='utf-8') as f:
            data = json.load(f)

        map_data = data.get('map', [])

        # Если map_data – список строк, превращаем в список списков
        if map_data and isinstance(map_data[0], str):
            self.grid = [list(row) for row in map_data]
        else:
            self.grid = map_data

        self.current_file = file_path
        self.has_changes = False

        self.canvas.set_grid(self.grid)

        filename = os.path.basename(file_path)
        pygame.display.set_caption(f"Map Editor – {filename}")

        print(f"[Успех] Загружен уровень: {file_path}")
        print(f"    Размер: {len(self.grid[0])}x{len(self.grid)}")

        return True

    except Exception as e:
        print(f"[Ошибка] Не удалось загрузить уровень: {e}")
        return False

def save_level(self, file_path=None):
    if file_path is None:
        file_path = self.current_file

    if not file_path or not self.grid:
        return False

    try:
        # Загружаем существующие данные
        data = {}
        if os.path.exists(file_path):
            with open(file_path, 'r', encoding='utf-8') as f:
                data = json.load(f)

        # Обновляем карту
        data['map'] = self.grid

        # =====
        # РУЧНОЕ ФОРМАТИРОВАНИЕ
        # =====
        # Форматируем map вручную
        map_lines = []
        for row in self.grid:

```

```

        json_row = json.dumps(row, ensure_ascii=False)
        map_lines.append(f"    {json_row}")
    map_json = "[" + "\n" + ",\n".join(map_lines) + "\n    ]"

    # Форматируем остальные данные
    meta_data = {k: v for k, v in data.items() if k != 'map'}
    meta_json = json.dumps(meta_data, indent=4, ensure_ascii=False)

    # Собираем финальный JSON
    final_json = "{\n" + f"    \"map\": {map_json},\n" + meta_json[2:]

    with open(file_path, 'w', encoding='utf-8') as f:
        f.write(final_json)

    self.has_changes = False
    self._clear_backups()

    print(f"[Сохранено] {file_path}")
    return True

except Exception as e:
    print(f"[Ошибка] Сохранение: {e}")
    return False

def _open_dialog(self):
    if self.has_changes and self.current_file:
        self.dialog_active = True
        self.dialog_choice = None
    else:
        self.running = False

def _draw_dialog(self):
    overlay = pygame.Surface((WINDOW_WIDTH, WINDOW_HEIGHT))
    overlay.set_alpha(180)
    overlay.fill((0, 0, 0))
    self.screen.blit(overlay, (0, 0))

    dialog_rect = pygame.Rect(
        WINDOW_WIDTH // 2 - 220,
        WINDOW_HEIGHT // 2 - 80,
        440, 160
    )
    pygame.draw.rect(self.screen, (50, 50, 60), dialog_rect)
    pygame.draw.rect(self.screen, (100, 100, 120), dialog_rect, 2)

    font = pygame.font.Font(None, 24)
    font_small = pygame.font.Font(None, 18)

    title = font.render("Сохранить изменения?", True, (255, 255, 255))
    title_rect = title.get_rect(center=(dialog_rect.centerx, dialog_rect.y + 30))
    self.screen.blit(title, title_rect)

    y = dialog_rect.y + 70
    options = [
        ("[Y] Да, сохранить и выйти", (dialog_rect.x + 30, y)),
        ("[N] Нет, выйти без сохранения", (dialog_rect.x + 30, y + 25)),
        ("[ESC] Отмена", (dialog_rect.x + 30, y + 50)),
    ]

```

```

for text, pos in options:
    rendered = font_small.render(text, True, (200, 200, 200))
    self.screen.blit(rendered, pos)

def _handle_events(self):
    for event in pygame.event.get():
        if self.dialog_active:
            if event.type == pygame.KEYDOWN:
                if event.key == pygame.K_y:
                    self.dialog_choice = 'save'
                    self.dialog_active = False
                elif event.key == pygame.K_n:
                    self.dialog_choice = 'discard'
                    self.dialog_active = False
                elif event.key == pygame.K_ESCAPE:
                    self.dialog_choice = 'cancel'
                    self.dialog_active = False
            continue

        if event.type == pygame.QUIT:
            self._open_dialog()
            continue

        # ПАНЕЛЬ ИНСТРУМЕНТОВ
        if event.type == pygame.MOUSEBUTTONDOWN:
            mx, my = event.pos
            if self.tools_panel.handle_click(mx, my):
                self.current_tool = self.tools_panel.get_selected_tool()
                if self.current_tool != 'select':
                    self.selection.start_x = None
                    self.selection.start_y = None
                    self.selection.end_x = None
                    self.selection.end_y = None
            continue

        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_ESCAPE:
                self._open_dialog()
                continue

            if event.key == pygame.K_s and (event.mod & pygame.KMOD_CTRL):
                if self.current_file:
                    self.save_level(self.current_file)
                continue

            if event.key == pygame.K_0 and (event.mod & pygame.KMOD_CTRL):
                self.canvas._center_view()

            if event.key == pygame.K_DELETE:
                if self.current_tool == 'select' and self.selection.get_selection():
                    self.selection.clear()

        elif event.type == pygame.MOUSEBUTTONDOWN:
            mx, my = event.pos

            if self.toolbar.rect.collidepoint(mx, my):
                if event.button == 4:
                    self.toolbar.scroll(-30)
                    continue

```

```

        elif event.button == 5:
            self.toolbar.scroll(30)
            continue

    if self.toolbar.handle_click(mx, my):
        symbol = self.toolbar.get_selected_symbol()
        self.selected_symbol = symbol
        if self.current_tool == 'select' and self.selection.get_selection():
            self.selection.fill(symbol)
        continue

    if self.canvas.rect.collidepoint(mx, my):
        if event.button == 4:
            self.canvas.zoom_in()
            continue
        elif event.button == 5:
            self.canvas.zoom_out()
            continue

    if event.button == 2:
        self.canvas.start_drag(mx, my)
        continue

    cell = self.canvas.get_cell_at(mx, my)
    if not cell:
        continue

    x, y = cell

    if self.current_tool == 'brush' and event.button == 1:
        self.brush.apply(x, y)
    elif self.current_tool == 'eraser' and event.button == 1:
        self.eraser.apply(x, y)
    elif self.current_tool == 'select' and event.button == 1:
        self.selection.start(x, y)

elif event.type == pygame.MOUSEBUTTONUP:
    if event.button == 2:
        self.canvas.end_drag()

    if self.current_tool == 'select' and event.button == 1:
        self.selection.end()

elif event.type == pygame.MOUSEMOTION:
    mx, my = event.pos
    self.canvas.update_drag(mx, my)

    cell = self.canvas.get_cell_at(mx, my)

    if pygame.mouse.get_pressed()[0] and cell:
        x, y = cell
        if self.current_tool == 'brush':
            self.brush.apply(x, y)
        elif self.current_tool == 'eraser':
            self.eraser.apply(x, y)
        elif self.current_tool == 'select':
            self.selection.update(x, y)

    if pygame.mouse.get_pressed()[2] and cell:

```

```

        x, y = cell
        self.eraser.apply(x, y)

    symbol = None
    if cell:
        x, y = cell
        if 0 <= y < len(self.grid) and 0 <= x < len(self.grid[0]):
            symbol = self.grid[y][x]
        self.info_panel.update(cell, symbol)
        self.canvas.update_hover(mx, my)

def _draw(self):
    self.screen.fill(COLORS['background'])

    self.canvas.draw(self.screen)

    if self.current_tool == 'select':
        self.selection.draw(self.screen)

    # Обновляем статус в info_panel
    self.info_panel.update_status(
        self.current_tool,
        self.selected_symbol,
        self.has_changes
    )

    self.info_panel.draw(self.screen)
    self.toolbar.draw(self.screen)
    self.tools_panel.draw(self.screen)

    if self.dialog_active:
        self._draw_dialog()

    pygame.display.flip()

def run(self):
    while self.running:
        self._handle_events()

        if not self.dialog_active and self.dialog_choice is not None:
            if self.dialog_choice == 'save':
                if self.current_file:
                    self.save_level(self.current_file)
                self.running = False
            elif self.dialog_choice == 'discard':
                self.running = False
            elif self.dialog_choice == 'cancel':
                self.dialog_choice = None

        self._draw()
        self.clock.tick(60)

    pygame.quit()

```